

Create a system test plan

This task consists of two parts: A high-level test plan for the control module, and on a lower level writing system test cases. This is followed by a RAID log to make assumptions explicit instead of implicit, and closed off by the list of improvement opportunities that could extend & polish up this work.

HL system test plan

The "below system" is the VISTA-20P control module, unintegrated with other components.

There's no knowledge about internal testing functions, so it will have to be tested in a HIL (Hardware in a loop) style. This means that a HIL testbench will be made (if one doesn't already exist), which both powers the module, and simulates necessary I/O. Furthermore, the testbench shall have the functionality to control the VISTA control module's state. This is done, so the sequencing of the tests makes no impact on the result of the testing (e.g.: an arming signal shall lead to very different behaviour in a disarmed state vs in an alarm state).

The created tests shall be traceable to a specific requirement. To support easy communication with non-technical stakeholders, the test plan shall consist of 3 pre-defined test-suites, on an intermediate abstraction level.

The created tests shall be e2e in their nature, as the available specifications don't specify internal architecture and internal test interfaces either (this is in accordance with principle of the V-model).

Useful semantics:

- Zone: a collection of detection devices, that together define a boundary where monitoring/protection is desired. [Zone types](#)
- Partition: Two independent areas of protection for use by independent users.
- Common partition: A set of zones shared by both partition (e.g. lobby area)

HIL testbench capability levels

Depending on the volume of the units/systems to be tested, the HIL testbench shall be sufficiently automated. Roughly speaking (without performing proper financial analysis), the following order is advised (with some further pointers behind):

- Dedicated fixture for the device under test - to protect against the 'fell on the ground' and similar type of damages & risks.
- Writing drivers conforming the VISTA product family i/f specs (this is a prerequisite for state management - this ranges from the levels of 'enable digital pin 5 HIGH', to the levels of 'simulate door-open message from door-sensor'. The benefit here is that this

reduces cognitive load on the testers, thereby improving test reliability. It also gets progressively faster to test this way)

- Fully automated smoke-test suite (Depends on the size of the test-suite, but for example NI hardware switching following a C-code is faster to the degree that it can probably perform the entire suite in a time an operator manually tests it with a signal generator & oscilloscope combo. Not to mention reliability gains).
- Fully automated test-suite (this is a minor increment compared to a fully automated smoke-test suite)
- Interface attach/detach: Brackets (e.g. 3D printed) with pre-installed cables and test-pins, for faster power & interface attach/detach (screwing in cables is especially time-consuming - which offsets quite early the time it takes to design a pin-based fixture interface)
- *Test subassemblies before assembly (whether this is worth it is highly dependent on the quality of the subassemblies - so this is a bit more situational)*

Test case definitions

Given the high likelihood that the testing work will need to be repeated, the test case definitions shall form the backbone of the documentation of the work.

Since the overall knowledge regarding the product is low in the test-team, the system test case specifications shall follow an AAA (Arrange, Act, Assert) format. This was selected over the BDD (Given, When, Then) format, because the current level of the drivers from the proposed HIL testbench capability is virtually zero, and I haven't got confirmation that I shall expect this capability to be built. This means that test-team will need to ensure that both the drivers, and the test-cases exist (former is coding work, latter is plain English). To test the system both need to exist, but the task of defining and documenting a mature mapping between the English keywords and the code functions can be avoided by selecting AAA over BDD. This is a trade-off between test-case development speed and ideal architecture.

I can envision a driver layer to be handy for a development team(s) too, maybe already existing. If these drivers already exist, I'd certainly give an attempt to hook BDD-style tests into that driver-layer (if technologically feasible), as that reduces the scope of testing quite substantially, even when accounting for the additional task of the keyword-code mapping.

In either case, I want to avoid the scenario, when the test-team silently absorbs the cost of developing this system interfacing capability (the drivers), as this runs the risk of this being perceived as a slow-down during testing.

(Regarding the phrasing: I am well-aware that the team in this assignment means only me, and there's no further corporate context, but policies and guidelines shall have a more abstract, person-independent target group to be maintainable over the timeframe of quarters)

Test suites

The defined test-suites shall act as an intermediate abstraction level between the level "does the system work? Yes-no style", and the "which exact test are failing?". There are some already well-defined abstractions in the feature description, namely:

- **"Features/system functions:"**
- **"Valuable End-User Features/functions:"**

There's one more, that almost certainly will get asked for: a smoke-test set. These shall contain the test for the most important features only.

Traceability

Traceability between requirements and test suites:

- Features/system functions: 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15, 16
- Valuable End-User Features/functions: 17, 18, 19, 20, 21, 22, 23, 24, 25, 26
- Smoke-test: 02 (only keypad support), 04, 08, 09, 15 (falshing for alarm), 21 (as an override in case of i/o malfucntions)

System test case specification

The system test case specifications are outlined below. For the sake of interpretability, the requirement inspiring each test case have been left there, however the source of truth shall be the traceability matrix in the section below.

For the sake of this assignment, I've written various test case approaches, to showcase that the approach is fully tailorable to the available tools, expertise & level of automation around.

Test cases

- 01 - IP alarm reporting and uploading/ downloading capability for Internet and Intranet use via iGSMV4G, 7845i-ENT, GSMV4G or GSMX4G
 - T-01 Test alarm forwarding:
 - Arrange
 - Connect an iGSMV4G module to the SUT (system under test)
 - Ensure that the SUT recognizes & is configured for "ip alarm reporting"
 - Put the SUT in armed mode
 - Act
 - Initiate alarm in SUT (e.g. simulate open door)
 - Assert
 - Verify in the iGSMV4G module's web-based maintenance interface that the alarm has been forwarded
 - T-02 Test uploading/downloading functionality
 - Prerequisite

- Site definition (set of zones), and Modified site definition (similar set of zones, but one zone is changed) as upload-able code
- Arrange
 - Connect the Control panel SUT with a compatible "Internet/Intranet Communication device" (e.g. and iGSMV4G)
 - Ensure that Site definition is loaded on the Control panel SUT
 - Go through the workflow specified in the [specs](#)

To perform upload/download functions:

 1. Connect the computer to the Internet and start the Compass downloading software.
 2. Open the control's account, then select the Communications function and click the Connect button.
 3. At the Connect screen, check that the control's MAC address is entered and the TCP/IP checkbox is checked.
 4. Click Connect. The Internet connection to the control is made automatically via AlarmNet.
 5. Once connected, use the Compass downloading software as normal to perform upload/download functions.
- Act
 - Upload the Modified site definition via the Compass downloading software
- Assert
 - Check that the zone have changed
- Supports four graphic touchscreen keypads
 - T-03 Test four graphic touchscreens
 - Arrange
 - Ensure that the system is in a disarmed state
 - Connect 4 keypads (n.b. you don't need the latest touchscreen models, the controller uses the VISTA family I/F, which haven't changed between the touchscreen and the normal keypads)
 - Ensure that all of the keypad 2-4 are enabled by having a unique addresses
 - Act
 - Cycle through arm-disarm with all 4 keypads
 - Assert

- The armed led was lit up with all 4 keypads



- Wireless keys can be programmed without using zones
 - T-04: Wireless keys without zones
 - **TODO: it is unclear from unclear from additionally googled specs how this can be done** (I use google for this exercise as my KMS). The problem is that based on the [specs](#), a wireless key's button functions as a loop, and the loops need to be bundled into zones so a zone type can be assigned to them? Very well could be that I am reading old specs / I am misunderstanding this aspect here. With that being said, here's a high level test that could make this behaviour observable, than the devs & architects can decide if this is intended or not..
 - Arrange
 - Ensure that the control panel is unarmed & the wireless key is configured, but no zones are used
 - Act
 - Press arm on the wireless key
 - Assert
 - The system arms
- Eight on-board hardwired zones standard (15 when Zone Doubling feature is used)
 - 40 hardwire expansion zones
 - T-05: Stress-test hardwire expansion zones
 - Arrange
 - Configure the SUT to have 40 hardwire expansion zones (needs additional HW/ a HIL testbench)
 - Arm the system
 - Act
 - Send in continuous data at the maximum rate for all 40 zones

- Assert
 - Ensure that the system keeps up with the load (no lagging, no restarting, no glitching, no unexpected behaviour)
- T-06: 41th zone fails *Note: defining expected to fail tests are only worth for functionality that needs to be 100% fault-proofed, as on this level executing such tests is much more expensive than on a lower level.*
 - Arrange
 - Reuse T-05 40 hardwire configuration
 - Act
 - Try to configure a 41th zone
 - Assert
 - The system shall display an error and deny the configuration of the 41th hardwire expansion zone
- 40 wireless expansion zones
 -
- Two low current on-board trigger outputs
- 100 Event Log viewable at system keypads with time/date stamp
 - T-07: Too low power is visible in the logs *This is again corner cutting as half of the feature gets missed, but it demonstrates how a single test can be used to scoop up the main functionality for multiple requirements - we've used such things in the past to evaluate system readiness*
 - Arrange
 - Arm the system
 - Act:
 - Modulate the power source until there is too low current on-board
 - Assert:
 - The alarm was triggered
 - This is visible in the event log too

And so on and so forth (doing this to cover all requirements would take quite some additional time, so I stopped here for now)

Traceability

All requirements is tested by relations recorded in a machine readable format below.

RQ_id: [array of test_ids testing the given RQ_id]

```
{
01: [T-01, T02],
02: [T-03],
03: [T-04],
04: [T-05, T-06],
07: [T-07],
```

```
08: [T-07]
```

```
}
```

Coverage

Strictly speaking we have 6 requirements covered (even though for RQ id 04 and RQ id 08 I am not happy with how well the existing tests are covering the requirements). Given that we have 22 RQs, we are at an $6 / 22 \approx 27\%$ coverage. That's barely enough to have a feel for the quality of the system.

RAID log

There were many ambiguous points during the completion of this assignment. Normally I'd go and ask around / consult the KMS of the company, but that was not an available option here. Therefore I've decided to maintain a RAID log for this task, so we can revisit decisions in case this steers off the direction the task was aimed at.

- 09/21 16:40 - Risk: There are now two set of features with the assignment linking both the pdf, then giving specs below.
 - Decided to treat the pdf as example, and the word-doc specs as authoritative in case there's a disagreement.
 - 17:03 - revisited this point, implications are better understood: There are indeed differences here, mostly scope-wise the word-doc doesn't include power consumption ("electrical"), standards/compliance ("listings"), and "security dealer features" related features. This seems scoping-only, therefore I didn't amend this decision, the word doc is still authoritative.
- 09/21 16:50 - Decision: There are quite many features, that warrant an intermediate level of abstraction. These are going to supply the basis of our test suites (so in that case the test plan will be: execute the following test-suites)
- 09/21 16:57 - Issue (Resolved): There are no id-s for the VISTA-20P requirements. I (MarkL) have decided that I need to have requirement id-s, so we can do traceability. No time to do fancy things with the id-s, so I'll just number them in a 01-99 style.
 - Delegated off to AI with reminder to "The final set should be jira-importable"
 - [VISTA-Requirements-with-id.csv](#)
 - Some id-s are unpopulated, because AI produced not fully reliable output - needed to clean that up.
- 09/21 ?? *exact time lost* - *I forgot to move this decision to the RAID log, and found it later in my text:*

Regarding what constitutes as a smoke-test, in absence of other people setting the priorities (such as a dedicated PO), I've made the decision myself, based on the guiding principle of "up to my best knowledge, is this essential feature for an alarm-control system, or is this an enhancement over the basic premise of 'local alert when break-in'?".

- 09/21 17:30 - Assumption: "below system" is a bit ambiguous and left for interpretation. Since there is no-one to ask (sent q to Nearfield via Liam), I've decided that the "below system" refers to the VISTA-20P control module as a standalone - rolling off from the production line (so not integrated with anything yet) - system.
 - btw, important that I don't have domain knowledge with alarm systems, and it shows. Decided to go for a top-down approach instead of more googling.
- 09/21 18:30 - Assumption: there's not much context regarding what's the goal of our testing here, in what phase of product development. For that reason I've assumed that this is mid-development, when the first 10-100s of prototypes are getting created, but the design can't be fundamentally altered anymore.

Improvement opportunities

- HIL testing: with more context I can probably champion/lead the entire HIL development. The assignment (understandably) lacks the data to do that in a grounded manner & I doubt that would be the way I could make the biggest impact at Nearfield, so let's not go there! ;)
- Improve the coverage: Finish the test cases. Also a bit better documentation could be given than citing the specs document as-is, documentation of why am I testing each feature the way I am testing it. I've seen the latter aspect biting when it comes to maintainability on the years long horizon, but only softly, so it's a choice about taking out tech-debt.
 - Assuming the current quality, it would take 1 man-day to finish it up.
 - Calculation:
Writing the current set of test cases (incl. finding & understanding the additional documentation) took 2 hours (tracker says 1:56 to be exact). We've covered 5 requirements in full from 22 requirements. Extrapolating based on that, writing the full test suite would take another 7 hours. For the ease of planning, and giving a margin for context switching, 1d shall be used for planning reasons

$$\text{Full time it takes: } 2\text{h} / (5 / 22) = 8.8\text{h}$$

$$\text{Remaining time it takes: } 8.8\text{h} - 2\text{h} = 6.8\text{h}$$
- Populate the test cases into jira (preferably using the Xray plugin): I have a nice pattern in my mind for configuring / structuring jira with Xray, so coverage reports and traceability matrices can be generated automatically. Unfortunately I don't own a licensed environment where I can replicate that and neatly show it (NDA..)
- General touch-up on the text quality. I haven't run dedicated grammar check over this text